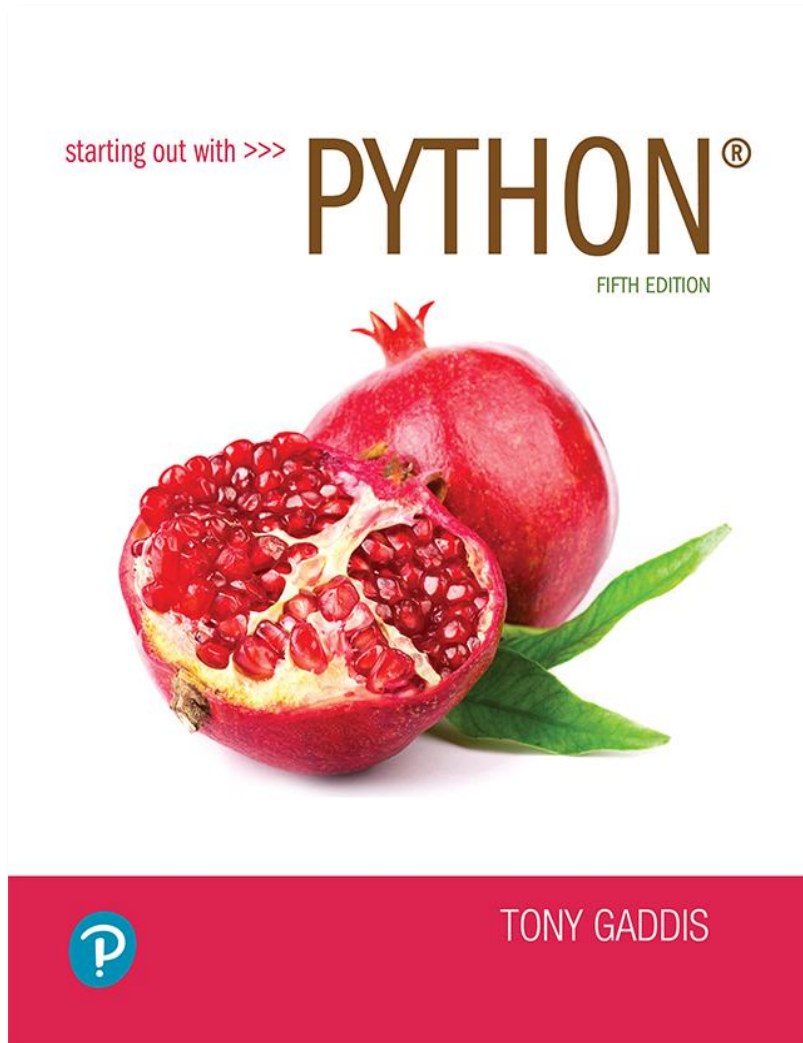


Starting out with Python

Fifth Edition



Chapter 5

Functions

Topics (1 of 2)

- Introduction to Functions
- Defining and Calling a Void Function
- Designing a Program to Use Functions
- Local Variables
- Passing Arguments to Functions
- Global Variables and Global Constants
- Turtle Graphics: Modularizing Code with Functions

Topics (2 of 2)

- Introduction to Value-Returning Functions: Generating Random Numbers
- Writing Your Own Value-Returning Functions
- The `math` Module
- Storing Functions in Modules

Introduction to Functions (1 of 2)

- Function: group of statements within a program that perform as specific task
 - Usually one task of a large program
 - Functions can be executed in order to perform overall program task
 - Known as *divide and conquer* approach
- Modularized program: program wherein each task within the program is in its own function



Introduction to Functions (2 of 2)

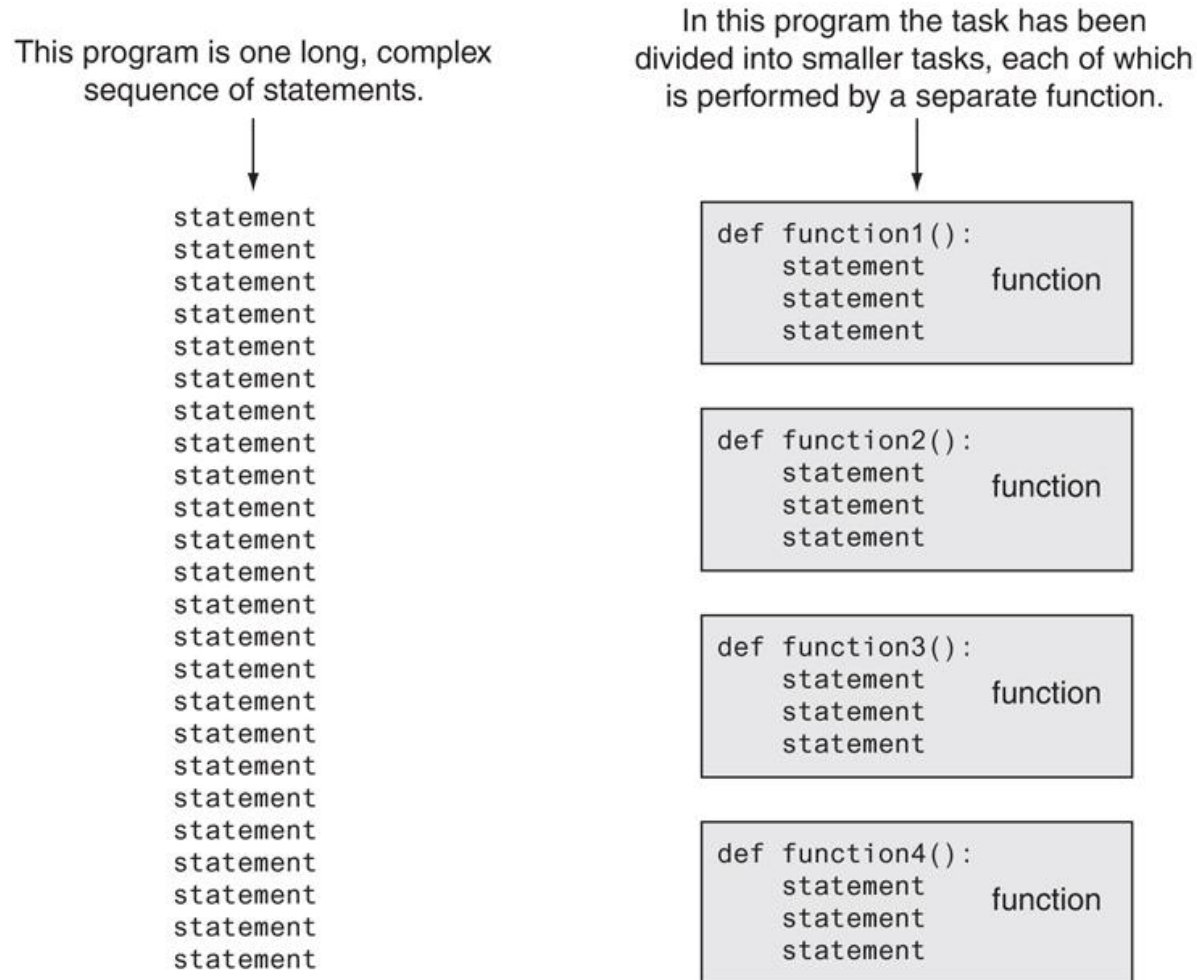


Figure 5-1 Using functions to divide and conquer a large task

Benefits of Modularizing a Program with Functions

- The benefits of using functions include:
 - Simpler code
 - Code reuse
 - write the code once and call it multiple times
 - Better testing and debugging
 - Can test and debug each function individually
 - Faster development
 - Easier facilitation of teamwork
 - Different team members can write different functions

Void Functions and Value-Returning Functions

- A void function:
 - When you call a void function, it simply executes the statements it contains and then terminates.
- A value-returning function:
 - Executes the statements it contains, and then it returns a value back to the statement that called it.
 - The `input`, `int`, and `float` functions are examples of value-returning functions.



Checkpoint

- 5.1 What is a function?
- 5.2 What is meant by the phrase “divide and conquer”?
- 5.3 How do functions help you reuse code in a program?
- 5.4 How can functions make the development of multiple programs faster?
- 5.5 How can functions make it easier for programs to be developed by teams of programmers?

Defining and Calling a Function (1 of 5)

- Functions are given names
 - Function naming rules:
 - Cannot use keywords as a function name
 - Cannot contain spaces
 - First character must be a letter or underscore
 - All other characters must be a letter, number or underscore
 - Uppercase and lowercase characters are distinct

Defining and Calling a Function (2 of 5)

- Function name should be descriptive of the task carried out by the function
 - Often includes a verb
- Function definition: specifies what function does

```
def function_name() :  
    statement  
    statement
```

Defining and Calling a Function (3 of 5)

- Function header: first line of function
 - Includes keyword `def` and function name, followed by parentheses and colon
- Block: set of statements that belong together as a group
 - Example: the statements included in a function

Example 1

```
def message():  
    print('I am Mohamed,')  
    print('Your best friend .')
```

```
message()
```

Defining and Calling a Function (4 of 5)

- Call a function to execute it
 - When a function is called:
 - Interpreter jumps to the function and executes statements in the block
 - Interpreter jumps back to part of program that called the function
 - Known as function return

Defining and Calling a Function (5 of 5)

- main function: called when the program starts
 - Calls other functions when they are needed
 - Defines the *mainline logic* of the program

Example 2

```
def main():  
    print('I have a message for you,')  
    message()  
    print('Goodbye !,')  
def message():  
    print('I am Mohamed,')  
    print('Your best friend .')  
  
main()
```

Indentation in Python

- Each block must be indented
 - Lines in block must begin with the same number of spaces
 - Use tabs or spaces to indent lines in a block, but not both as this can confuse the Python interpreter
 - IDLE automatically indents the lines in a block
 - Blank lines that appear in a block are ignored



Checkpoint

- 5.6 A function definition has what two parts?
- 5.7 What does the phrase “calling a function” mean?
- 5.8 When a function is executing, what happens when the end of the function’s block is reached?
- 5.9 Why must you indent the statements in a block?

Designing a Program to Use Functions

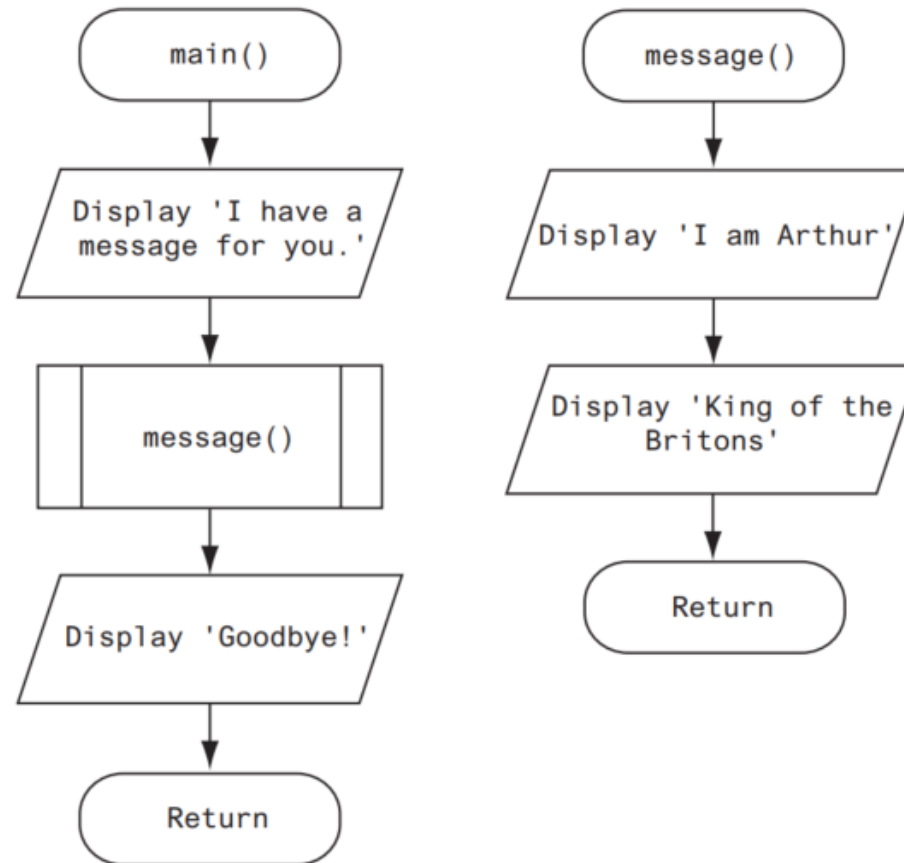
(1 of 5)

- In a flowchart, function call shown as rectangle with vertical bars at each side
 - Function name written in the symbol
 - Typically draw separate flow chart for each function in the program
 - End terminal symbol usually reads `Return`
- Top-down design: technique for breaking algorithm into functions

Designing a Program to Use Functions

(2 of 5)

Figure 5-9 Flowchart for Program 5-2



Designing a Program to Use Functions

(3 of 5)

The process of top-down design is performed in the following manner:

- The overall task that the program is to perform is broken down into a series of subtasks.
- Each of the subtasks is examined to determine whether it can be further broken down into more subtasks. This step is repeated until no more subtasks can be identified.
- Once all of the subtasks have been identified, they are written in code.

This process is called top-down design because the programmer begins by looking at the topmost level of tasks that must be performed and then breaks down those tasks into lower levels of subtasks.

Designing a Program to Use Functions

(4 of 5)

- Hierarchy chart: depicts relationship between functions
 - AKA *structure chart*
 - Box for each function in the program, Lines connecting boxes illustrate the functions called by each function
 - Does not show steps taken **inside a function**
- Use `input` function to have program wait for user to press enter

Designing a Program to Use Functions

(5 of 5)

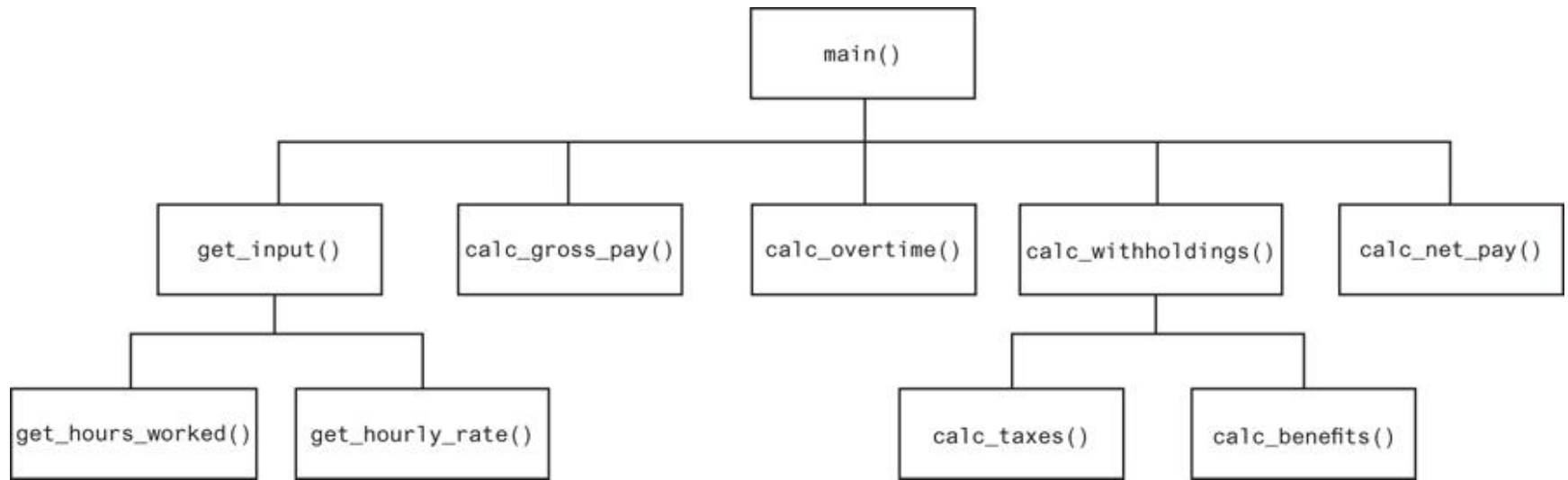
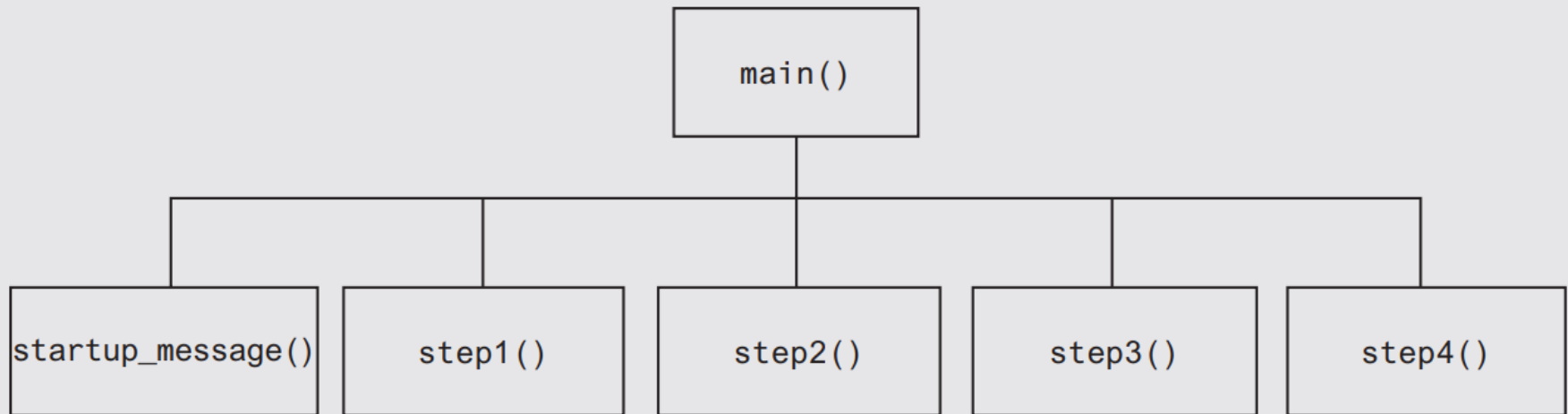


Figure 5-10 A hierarchy chart

This design shows the steps inside the function



```
def main():
    startup_message()
    input('Press enter to see step 1: ')
    step1()
    input('Press enter to see step 2: ')
    step2()
    input('Press enter to see step 3:')
    step3()
    input('Press enter to see step 4:')
    step4()
    input('Press enter to see step 5:')
    step5()

def startup_message():
    print('this program shows how to make tea')
    print('the are 5 simple steps')
    print()

def step1():
    print('Step1: Heat water until it boils')
    print()

def step2():
    print('Step 2: Put tea leaves in a cup')
    print()

def step3():
    print('Step 3: Pour the hot water into the cup')
    print()

def step4():
    print('Step 4: Wait for a few minutes to let the tea mix with the water ')
    print()

def step5():
    print('Step 5: Add sugar or milk if you like, then enjoy your tea!')

main()
```


Using the `pass` Keyword

- You can use the `pass` keyword to create empty functions
- The `pass` keyword is ignored by the Python interpreter
- This can be helpful when designing a program

```
def step1():  
    pass  
  
def step2():  
    pass
```

Local Variables (1 of 2)

- Local variable: variable that is assigned a value inside a function
 - Belongs to the function in which it was created
 - Only statements inside that function can access it, error will occur if another function tries to access the variable
- Scope: the part of a program in which a variable may be accessed
 - For local variable: function in which created

Local Variables (2 of 2)

- Local variable cannot be accessed by statements inside its function which precede its creation
- Different functions may have local variables with the same name
 - Each function does not see the other function's local variables, so no confusion

Example

```
def main():  
    get_name()  
    print(f'Hello {name}')  Error  
def get_name():  
    name=input("Enter your name: ")  
  
main()
```

Example

```
def main():  
    hayat_market()  
    jubba_market()  
def hayat_market():  
    branch=3  
    print(f'hayat market has {branch} branches')  
def jubba_market():  
    branch=2  
    print(f'Jubba market has {branch} branches')  
  
main()
```



Checkpoint

- 5.10 What is a local variable? How is access to a local variable restricted?
- 5.11 What is a variable's scope?
- 5.12 Is it permissible for a local variable in one function to have the same name as a local variable in a different function?

Passing Arguments to Functions (1 of 4)

Sometimes it is useful not only to call a function, but also to send one or more pieces of data into the function. Pieces of data that are sent into a function are known as *arguments*. The function can use its arguments in calculations or other operations.

If you want a function to receive arguments when it is called, you must equip the function with one or more parameter variables. A *parameter variable*, often simply called a *parameter*, is a special variable that is assigned the value of an argument when a function is called. Here is an example of a function that has a parameter variable:

```
def show_double(number):  
    result = number * 2  
    print(result)
```

Passing Arguments to Functions (2 of 4)

```
def main():  
    value = 5  
    show_double(value)  
  
def show_double(number):  
    result = number * 2  
    print(result)
```

A diagram consisting of a horizontal line segment followed by a vertical line segment ending in a downward-pointing arrowhead, indicating the flow of the argument 'value' from the 'show_double' call in the 'main' function to the 'number' parameter in the 'show_double' function definition.

Figure 5-13 The `value` variable is passed as an argument

Passing Arguments to Functions (3 of 4)

- Parameter variable: variable that is assigned the value of an argument when the function is called
 - The parameter and the argument reference the same value
 - General format:
 - `def function_name(parameter) :`
 - Scope of a parameter: the function in which the parameter is used

Example

```
def main():  
    print("This program displays the conversion of miles to kilometers")  
    print("The formula is 1 mile is equal to 1.60934 kilometers")  
    print()  
    miles = float(input('Enter the miles you needed: '))  
    miles_to_km(miles)  
  
def miles_to_km(value):  
    kilometers = value * 1.60934  
    print(f'{value} miles is equal to {kilometers:.2f} kilometers')  
  
main()
```

Passing Multiple Arguments (1 of 2)

- Python allows writing a function that accepts multiple arguments
 - Parameter list replaces single parameter
 - Parameter list items separated by comma
- Arguments are passed *by position* to corresponding parameters
 - First parameter receives value of first argument, second parameter receives value of second argument, etc.

Example

```
def main():  
    print("the sum of 12 and 45 is ")  
    show_sum(12,45)  
def show_sum(num1,num2):  
    result=num1+num2  
    print(result)  
  
main()
```

Example

```
def main():  
    num1=int(input("Enter number1:"))  
    num2=int(input("Enter number2:"))  
    calculate(num1,num2)  
    print("Process finished.....")
```

```
def calculate(a,b):  
    print("The addition is :", a + b)  
    print("The subtraction is :", a - b)  
    print("The product is :", a * b)  
    print("The division is :", a // b)  
main()
```

Making Changes to Parameters (1 of 3)

- Changes made to a parameter value within the function do not affect the argument
 - Known as *pass by value*
 - Provides a way for unidirectional communication between one function and another function
 - Calling function can communicate with called function

Example

```
def main():  
    value=99  
    print(f'the value is {value}')    change_me(value)  
    print(f'Back to main value {value}')def change_me(arg):  
    print('I am changing the value')  
    arg=0  
    print(f'Now the value is {arg}')  
main()
```

Making Changes to Parameters (2 of 3)

```
def main():  
    value = 99  
    print(f'The value is {value}.')  
    change_me(value)  
    print(f'Back in main the value is {value}.')
```

```
def change_me(arg):  
    print('I am changing the value.')  
    arg = 0  
    print(f'Now the value is {arg}.')
```

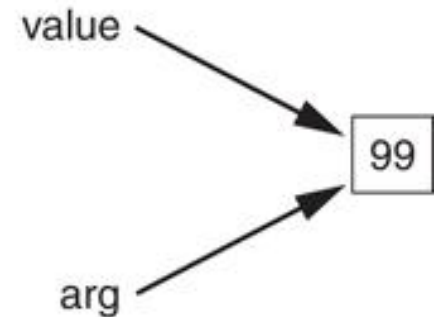


Figure 5-17 The `value` variable is passed to the `change_me` function

Making Changes to Parameters (3 of 3)

- Figure 5-18
 - The `value` variable passed to the `change_me` function cannot be changed by it

```
def main():  
    value = 99  
    print(f'The value is {value}.')  
    change_me(value)  
    print(f'Back in main the value is {value}.')
```

```
def change_me(arg):  
    print('I am changing the value.')  
    arg = 0  
    print(f'Now the value is {arg}.')
```

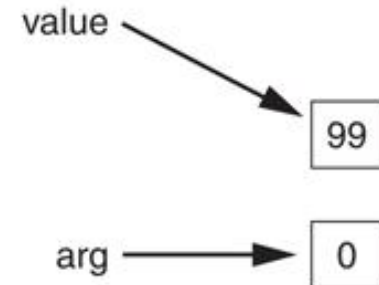


Figure 5-18 The `value` variable is passed to the `change_me` function

Keyword Arguments

- Keyword argument: argument that specifies which parameter the value should be passed to
 - Position when calling function is irrelevant
 - General Format:
 - `function_name(parameter=value)`
- Possible to mix keyword and positional arguments when calling a function
 - Positional arguments must appear first

Example

```
def main():  
    calculate_total_cost(apples=3.50, bread=2.00, milk=1.75, eggs=4.25)  
  
def calculate_total_cost(milk, eggs, apples, bread):  
    total=milk+eggs+apples+bread  
  
    print(f'The total cost of your shopping list is ${total:,.2f}.')  
  
main()
```

Global Variables and Global Constants

(1 of 3)

- Global variable: created by assignment statement written outside all the functions
 - Can be accessed by any statement in the program file, including from within a function
 - If a function needs to assign a value to the global variable, the global variable must be redeclared within the function
 - General format: `global variable_name`

Example

```
my_value = 10
def show_value():
    print(my_value)

show_value()
```

Example

```
number=0

def main():
    global number
    number=int(input('Enter a number: '))
    show_value()

def show_value():
    print(f'the number you entered is {number}')

main()
```

Example

```
my_value=10

def show_value():
    global my_value
    my_value=my_value+5
    print(f'the value is {my_value}')
show_value()
```

Global Variables and Global Constants

(2 of 3)

- Reasons to avoid using global variables:
 - Global variables making debugging difficult
 - Many locations in the code could be causing a wrong variable value
 - Functions that use global variables are usually dependent on those variables
 - Makes function hard to transfer to another program
 - Global variables make a program hard to understand

Global Variables and Global Constants

(3 of 3)

Most programmers agree that you should restrict the use of global variables, or not use them at all. The reasons are as follows:

- Global variables make debugging difficult. Any statement in a program file can change the value of a global variable. If you find that the wrong value is being stored in a global variable, you have to track down every statement that accesses it to determine where the bad value is coming from. In a program with thousands of lines of code, this can be difficult.
- Functions that use global variables are usually dependent on those variables. If you want to use such a function in a different program, most likely you will have to redesign it so it does not rely on the global variable.
- Global variables make a program hard to understand. A global variable can be modified by any statement in the program. If you are to understand any part of the program that uses a global variable, you have to be aware of all the other parts of the program that access the global variable.

In most cases, you should create variables locally and pass them as arguments to the functions that need to access them.

Global Constants

- Global constant: global name that references a value that cannot be changed
 - Permissible to use global constants in a program
 - To simulate global constant in Python, create global variable and do not re-declare it within functions

Marilyn works for **Integrated Systems, Inc.**, a software company that gives its employees **great extra benefits.**

- One of the advantages is a **bonus** that employees get **every three months (quarterly).**
- Another advantage is a **retirement savings plan**, where the company **adds money** to each employee's account.

The company helps employees **save for retirement** by adding **5% of their salary and bonuses** to their retirement savings.

Marilyn wants to create a **computer program** that will **calculate how much money the company adds to an employee's retirement savings** over a **whole year.**

She wants the program to:

1. **Show how much money is added** from the employee's **regular salary.**
2. **Show how much money is added** from the **quarterly bonuses.**

❑ Here is an **algorithm** step-by-step for the program

- ❖ Get the employee's annual gross pay.
- ❖ Get the amount of bonuses paid to the employee.
- ❖ Calculate and display the contribution for the gross pay.
- ❖ Calculate and display the contribution for the bonuses.

```
CONTRIBUTION_RATE=0.05
def main():
    pay=float(input('Enter the gross pay: '))
    bonus=float(input('Enter the amount of bonus: '))
    print()
    show_pay(pay)
    show_bonus(bonus)

def show_pay(gross):
    contribution=gross*CONTRIBUTION_RATE
    print(f'contribution for gross pay is ${contribution}')

def show_bonus(bonus):
    contribution=bonus*CONTRIBUTION_RATE
    print(f'contribution for bonus is ${contribution}')

main()
```



Checkpoint

- 5.18 What is the scope of a global variable?
- 5.19 Give one good reason that you should not use global variables in a program.
- 5.20 What is a global constant? Is it permissible to use global constants in a program?

Introduction to Value-Returning Functions: Generating Random Numbers

- void function: group of statements within a program for performing a specific task
 - Call function when you need to perform the task
- Value-returning function: similar to void function, returns a value
 - Value returned to part of program that called the function when function finishes executing

Standard Library Functions and the `import` Statement

Python, as well as most programming languages, comes with a *standard library* of functions that have already been written for you. These functions, known as *library functions*, make a programmer's job easier because they perform many of the tasks that programmers commonly need to perform. In fact, you have already used several of Python's library functions. Some of the functions that you have used are `print`, `input`, and `range`. Python has many other library functions. Although we won't cover them all in this book, we will discuss library functions that perform fundamental operations.

Some of Python's library functions are built into the Python interpreter. If you want to use one of these built-in functions in a program, you simply call the function. This is the case with the `print`, `input`, `range`, and other functions about which you have already learned. Many of the functions in the standard library, however, are stored in files that are known as *modules*. These modules, which are copied to your computer when you install Python, help organize the standard library functions. For example, functions for performing math operations are stored together in a module, functions for working with files are stored together in another module, and so on.

Standard Library Functions and the `import` Statement

In order to call a function that is stored in a module, you have to write an `import` statement at the top of your program. An `import` statement tells the interpreter the name of the module that contains the function. For example, one of the Python standard modules is named `math`. The `math` module contains various mathematical functions that work with floating-point numbers. If you want to use any of the `math` module's functions in a program, you should write the following `import` statement at the top of the program:

```
import math
```

This statement causes the interpreter to load the contents of the `math` module into memory and makes all the functions in the `math` module available to the program.

Because you do not see the internal workings of library functions, many programmers think of them as *black boxes*. The term “black box” is used to describe any mechanism that accepts input, performs some operation (that cannot be seen) using the input, and produces output. Figure 5-19 illustrates this idea.

Standard Library Functions and the import Statement

Figure 5-19 A library function viewed as a black box



We will first demonstrate how value-returning functions work by looking at standard library functions that generate random numbers and some interesting programs that can be written with them. Then, you will learn to write your own value-returning functions and how to create your own modules. The last section in this chapter comes back to the topic of library functions, and looks at several other useful functions in the Python standard library.

Generating Random Numbers

Random numbers are useful for lots of different programming tasks. The following are just a few examples.

- Random numbers are commonly used in games. For example, computer games that let the player roll dice use random numbers to represent the values of the dice. Programs that show cards being drawn from a shuffled deck use random numbers to represent the face values of the cards.
- Random numbers are useful in simulation programs. In some simulations, the computer must randomly decide how a person, animal, insect, or other living being will behave. Formulas can be constructed in which a random number is used to determine various actions and events that take place in the program.
- Random numbers are useful in statistical programs that must randomly select data for analysis.
- Random numbers are commonly used in computer security to encrypt sensitive data.

Generating Random Numbers (1 of 5)

- Random number are useful in a lot of programming tasks
- random module: includes library functions for working with random numbers
- Dot notation: notation for calling a function belonging to a module
 - **Format:** `module_name.function_name()`

Generating Random Numbers (2 of 5)

- randint function: generates a random number in the range provided by the arguments
 - Returns the random number to part of program that called the function
 - Returned integer can be used anywhere that an integer would be used

Generating Random Numbers (3 of 5)

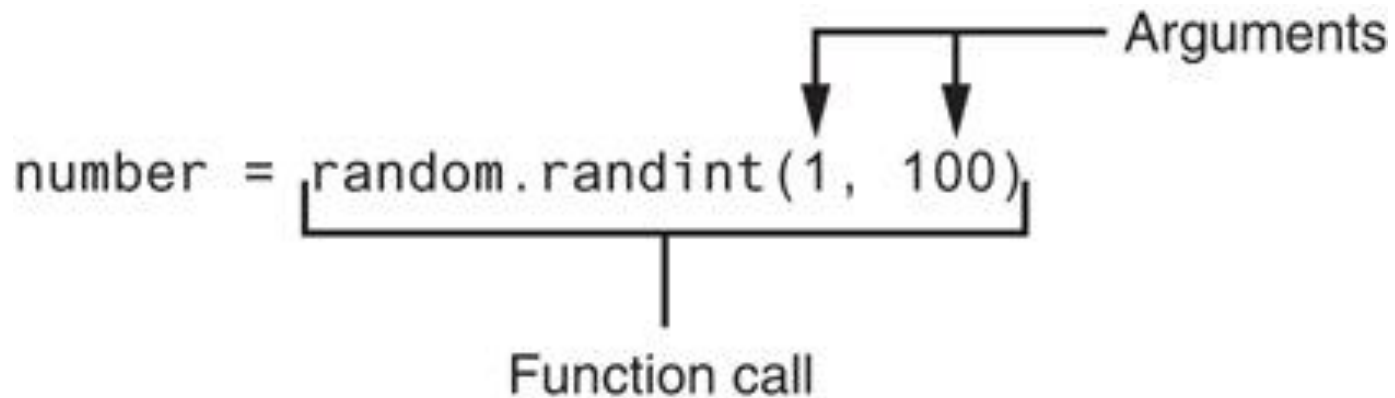
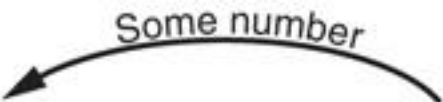


Figure 5-20 A statement that calls the random function

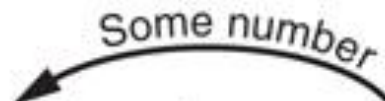
Generating Random Numbers (4 of 5)



```
number = random.randint(1, 100)
```

A random number in the range of
1 through 100 will be assigned to
the number variable.

Figure 5-21 The `random` function returns a value



```
print(random.randint(1, 10))
```

A random number in the range of
1 through 10 will be displayed.

Figure 5-22 Displaying a random number

Example

```
import random
def main():
    number = random.randint(1, 10)
    print(f'The number is {number}.')

main()
```



```
#This program displays five random  
# numbers in the range of 1 through 100.
```

```
import random  
def main():  
    for count in range(5):  
        number = random.randint(1, 10)  
        print(number)
```

```
main()
```

```
import random  
def main():  
    for count in range(5):  
        print(random.randint(1, 100))
```

```
main()
```

Example

```
import random
MIN = 0
MAX = 6
def main():
    again = 'y'
    while again == 'y' or again == 'Y':
        print('Rolling the dice . . .')
        print('Their values are:')
        print(random.randint(MIN, MAX))
        print(random.randint(MIN, MAX))

        again = input('Roll them again? (y = yes): ')

main()
```

Example

```
import random
HEADS = 1
TAILS = 2
TOSSES = 10
def main():
    for toss in range(TOSSES):
        if random.randint(HEADS, TAILS) == HEADS:

            print('Heads')
        else:

            print('Tails')

main()
```

The randrange, random, and uniform Functions

The standard library's `random` module contains numerous functions for working with random numbers. In addition to the `randint` function, you might find the `randrange`, `random`, and `uniform` functions useful. (To use any of these functions, you need to write `import random` at the top of your program.)

If you remember how to use the `range` function (which we discussed in Chapter 4), then you will immediately be comfortable with the `randrange` function. The `randrange` function takes the same arguments as the `range` function. The difference is that the `randrange` function does not return a list of values. Instead, it returns a randomly selected value from a sequence of values. For example, the following statement assigns a random number in the range of 0 through 9 to the `number` variable:

```
number = random.randrange(10)
```

The argument, in this case 10, specifies the ending limit of the sequence of values. The function will return a randomly selected number from the sequence of values 0 up to, but not including, the ending limit. The following statement specifies both a starting value and an ending limit for the sequence:

```
number = random.randrange(5, 10)
```

When this statement executes, a random number in the range of 5 through 9 will be assigned to `number`. The following statement specifies a starting value, an ending limit, and a step value:

```
number = random.randrange(0, 101, 10)
```

In this statement the `randrange` function returns a randomly selected value from the following sequence of numbers:

```
[0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
```

Both the `randint` and the `randrange` functions return an integer number. The `random` function, however, returns a random floating-point number. You do not pass any arguments to the `random` function. When you call it, it returns a random floating point number in the range of 0.0 up to 1.0 (but not including 1.0). Here is an example:

```
number = random.random()
```

The `uniform` function also returns a random floating-point number, but allows you to specify the range of values to select from. Here is an example:

```
number = random.uniform(1.0, 10.0)
```

In this statement, the `uniform` function returns a random floating-point number in the range of 1.0 through 10.0 and assigns it to the `number` variable.

5.8

Writing Your Own Value-Returning Functions

CONCEPT: A value-returning function has a `return` statement that returns a value back to the part of the program that called it.



VideoNote
Writing a
Value-Returning
Function

You write a value-returning function in the same way that you write a void function, with one exception: a value-returning function must have a `return` statement. Here is the general format of a value-returning function definition in Python:

```
def function_name():  
    statement  
    statement  
    etc.  
    return expression
```

One of the statements in the function must be a `return` statement, which takes the following form:

```
return expression
```

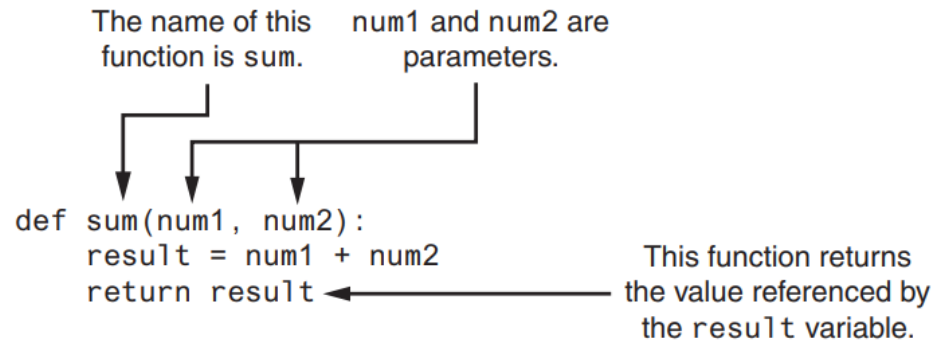
The value of the *expression* that follows the keyword `return` will be sent back to the part of the program that called the function. This can be any value, variable, or expression that has a value (such as a math expression).

Here is a simple example of a value-returning function:

```
def sum(num1, num2):  
    result = num 1 + num 2  
    return result
```

Figure 5-23 illustrates various parts of the function.

Figure 5-23 Parts of the function



The purpose of this function is to accept two integer values as arguments and return their sum. Let's take a closer look at how it works. The first statement in the function's block assigns the value of `num1 + num2` to the `result` variable. Next, the `return` statement executes, which causes the function to end execution and sends the value referenced by the `result` variable back to the part of the program that called the function. Program 5-21 demonstrates the function.

Program 5-21 (total_ages.py)

```
1  # This program uses the return value of a function.
2
3  def main():
4      # Get the user's age.
5      first_age = int(input('Enter your age: '))
6
7      # Get the user's best friend's age.
8      second_age = int(input("Enter your best friend's age: "))
9
10     # Get the sum of both ages.
11     total = sum(first_age, second_age)
12
13     # Display the total age.
14     print(f'Together you are {total} years old.')
15
16 # The sum function accepts two numeric arguments and
17 # returns the sum of those arguments.
18 def sum(num1, num2):
19     result = num1 + num2
20     return result
21
22 # Call the main function.
23 main()
```


How to Use Value-Returning Functions

Value-returning functions provide many of the same benefits as void functions: they simplify code, reduce duplication, enhance your ability to test code, increase the speed of development, and ease the facilitation of teamwork.

Because value-returning functions return a value, they can be useful in specific situations. For example, you can use a value-returning function to prompt the user for input, and then it can return the value entered by the user. Suppose you've been asked to design a program that calculates the sale price of an item in a retail business. To do that, the program would need to get the item's regular price from the user. Here is a function you could define for that purpose:

```
def get_regular_price():  
    price = float(input("Enter the item's regular price: "))  
    return price
```

Then, elsewhere in the program, you could call that function, as shown here:

```
# Get the item's regular price.  
reg_price = get_regular_price()
```

When this statement executes, the `get_regular_price` function is called, which gets a value from the user and returns it. That value is then assigned to the `reg_price` variable.

You can also use functions to simplify complex mathematical expressions. For example, calculating the sale price of an item seems like it would be a simple task: you calculate the discount and subtract it from the regular price. In a program, however, a statement that performs this calculation is not that straightforward, as shown in the following example. (Assume `DISCOUNT_PERCENTAGE` is a global constant that is defined in the program, and it specifies the percentage of the discount.)

```
sale_price = reg_price - (reg_price * DISCOUNT_PERCENTAGE)
```

At a glance, this statement isn't easy to understand because it performs so many steps: it calculates the discount amount, subtracts that value from `reg_price`, and assigns the result to `sale_price`. You could simplify the statement by breaking out part of the math expression and placing it in a function. Here is a function named `discount` that accepts an item's price as an argument and returns the amount of the discount:

```
def discount(price):  
    return price * DISCOUNT_PERCENTAGE
```

You could then call the function in your calculation:

```
sale_price = reg_price - discount(reg_price)
```

This statement is easier to read than the one previously shown, and it is clearer that the discount is being subtracted from the regular price. Program 5-22 shows the complete sale price calculating program using the functions just described.

Example

```
DISCOUNT_PERCENTAGE=0.20
```

```
def main():
```

```
    regular_price=get_regular_price()
```

```
    sales_price= regular_price-discount(regular_price)
```

```
    print(f'The sales price is ${sales_price}')
```

```
def get_regular_price():
```

```
    price=float(input("Enter the item's regular price: "))
```

```
    return price
```

```
def discount():
```

```
    return price*DISCOUNT_PERCENTAGE
```

```
main()
```

Returning Strings

- You can write functions that return strings
- For example:

```
def main():  
    user_name = get_name()  
    print("Your name is:", user_name)  
def get_name():  
    name = input('Enter your name: ')  
    return name  
  
main()
```

Returning Boolean Values

Python allows you to write *Boolean functions*, which return either `True` or `False`. You can use a Boolean function to test a condition, then return either `True` or `False` to indicate whether the condition exists. Boolean functions are useful for simplifying complex conditions that are tested in decision and repetition structures.

For example, suppose you are designing a program that will ask the user to enter a number, then determine whether that number is even or odd. The following code shows how you can make that determination:

```
number = int(input('Enter a number: '))
if (number % 2) == 0:
    print('The number is even.')
else:
    print('The number is odd.')
```

Let's take a closer look at the Boolean expression being tested by this `if-else` statement:

```
(number % 2) == 0
```

This expression uses the `%` operator, which was introduced in Chapter 2. This is called the remainder operator. It divides two numbers and returns the remainder of the division. So this code is saying, “If the remainder of number divided by 2 is equal to 0, then display a message indicating the number is even, or else display a message indicating the number is odd.”

Because dividing an even number by 2 will always give a remainder of 0, this logic will work. The code would be easier to understand, however, if you could somehow rewrite it to say, “If the number is even, then display a message indicating it is even, or else display a message indicating it is odd.” As it turns out, this can be done with a Boolean function. In this example, you could write a Boolean function named `is_even` that accepts a number as an argument and returns `True` if the number is even, or `False` otherwise. The following is the code for such a function:

```
def is_even(number):  
    # Determine whether number is even. If it is,  
    # set status to true. Otherwise, set status  
    # to false.  
    if (number % 2) == 0:  
        status = True
```



```
else:  
    status = False  
    # Return the value of the status variable.  
    return status
```

Then, you can rewrite the if-else statement so it calls the `is_even` function to determine whether number is even:

```
number = int(input('Enter a number: '))  
if is_even(number):  
    print('The number is even.')  
else:  
    print('The number is odd.')
```

Not only is this logic easier to understand, but now you have a function that you can call in the program anytime you need to test a number to determine whether it is even.

Returning Boolean Values

```
def is_even(number):  
    return number % 2 == 0  
number = int(input("Enter a number: "))  
if is_even(number):  
    print(f"{number} is even.")  
else:  
    print(f"{number} is odd.")
```


Returning Multiple Values

- In Python, a function can return multiple values
 - Specified after the `return` statement separated by commas
 - Format: `return expression1,`
`expression2, etc.`
 - When you call such a function in an assignment statement, you need a separate variable on the left side of the `=` operator to receive each returned value

Returning Multiple Values

```
def get_name():  
    first = input('Enter your first name: ')  
    last = input('Enter your last name: ')  
    return first, last  
frist_name, last_name = get_name()  
print(frist_name, last_name)
```

Returning None from a Function

Python has a special built-in value named `None` that is used to represent no value. Sometimes it is useful to return `None` from a function to indicate that an error has occurred. For example, consider the following function:

```
def divide(num1, num2):  
    return num1 / num2
```

The `divide` function takes two arguments, `num1` and `num2`, and returns the result of `num1` divided by `num2`. However, an error will occur if `num2` is equal to zero because division by zero is not possible. To prevent the program from crashing, we can modify the function to

determine whether `num2` is equal to 0 before we perform the division operation. If `num2` is equal to 0, we simply return `None`. Here is the modified code:

```
def divide(num1, num2):  
    if num2 == 0:  
        result = None  
    else:  
        result = num1 / num2  
    return result
```

Program 5-24 demonstrates how to call the `divide` function and use its return value to determine whether an error has occurred.

Program 5-24 (none_demo.py)

```
1  # This program demonstrates the None keyword.
2
3  def main():
4      # Get two numbers from the user.
5      num1 = int(input('Enter a number: '))
6      num2 = int(input('Enter another number: '))
7
8      # Call the divide function.
9      quotient = divide(num1, num2)
10
11     # Display the result.
12     if quotient is None:
13         print('Cannot divide by zero.')
14     else:
15         print(f'{num1} divided by {num2} is {quotient}.')
16
17     # The divide function divides num1 by num2 and
18     # returns the result. If num2 is 0, the function
19     # returns None.
20     def divide(num1, num2):
21         if num2 == 0:
22             result = None
23         else:
24             result = num1 / num2
25         return result
26
27     # Execute the main function.
28     main()
```

Program Output (with Input Shown in Bold)

Example

```
def main():
    num1=int(input("Entre num1: "))
    num2=int(input("Entre num2: "))
    result=divide(num1,num2)
    print(result)
def divide(number1,number2):
    if number2==0:
        result=None
    else:
        result=number1/number2
    return result

main()
```

Example

```
def main():
    num1 = int(input('Enter a number: '))
    num2 = int(input('Enter another number: '))
    quotient = divide(num1, num2)
    if quotient is None:
        print('Cannot divide by zero.')
    else:
        print(f'{num1} divided by {num2} is {quotient}.')
def divide(num1, num2):
    if num2 == 0:
        result = None
    else:
        result = num1 / num2
    return result

main()
```

The math Module (1 of 3)

- math module: part of standard library that contains functions that are useful for performing mathematical calculations
 - Typically accept one or more values as arguments, perform mathematical operation, and return the result
 - Use of module requires an `import math` statement

The math Module (2 of 3)

Table 5-2 Many of the functions in the `math` module

<code>math</code> Module Function	Description
<code>acos(x)</code>	Returns the arc cosine of <code>x</code> , in radians.
<code>asin(x)</code>	Returns the arc sine of <code>x</code> , in radians.
<code>atan(x)</code>	Returns the arc tangent of <code>x</code> , in radians.
<code>ceil(x)</code>	Returns the smallest integer that is greater than or equal to <code>x</code> .
<code>cos(x)</code>	Returns the cosine of <code>x</code> in radians.
<code>degrees(x)</code>	Assuming <code>x</code> is an angle in radians, the function returns the angle converted to degrees.
<code>exp(x)</code>	Returns e^x
<code>floor(x)</code>	Returns the largest integer that is less than or equal to <code>x</code> .
<code>hypot(x, y)</code>	Returns the length of a hypotenuse that extends from (0, 0) to (<code>x</code> , <code>y</code>).
<code>log(x)</code>	Returns the natural logarithm of <code>x</code> .
<code>log10(x)</code>	Returns the base-10 logarithm of <code>x</code> .
<code>radians(x)</code>	Assuming <code>x</code> is an angle in degrees, the function returns the angle converted to radians.
<code>sin(x)</code>	Returns the sine of <code>x</code> in radians.
<code>sqrt(x)</code>	Returns the square root of <code>x</code> .
<code>tan(x)</code>	Returns the tangent of <code>x</code> in radians.

Example

```
import math
def main():
    number = float(input('Enter a number: '))
    square_root = math.sqrt(number)
    print(f'The square root of {number} is {square_root}.')

main()
```

```
import math
print(math.ceil(12.3))
```

The math Module (3 of 3)

- The `math` module defines variables `pi` and `e`, which are assigned the mathematical values for π and e
 - Can be used in equations that require these values, to get more accurate results
- Variables must also be called using the dot notation
 - Example:

```
circle_area = math.pi * radius**2
```

Storing Functions in Modules (1 of 2)

- In large, complex programs, it is important to keep code organized
- Modularization: grouping related functions in modules
 - Makes program easier to understand, test, and maintain
 - Make it easier to reuse code for multiple different programs
 - Import the module containing the required function to each program that needs it

Storing Functions in Modules (2 of 2)

- Module is a file that contains Python code
 - Contains function definition but does not contain calls to the functions
 - Importing programs will call the functions
- Rules for module names:
 - File name should end in `.py`
 - Cannot be the same as a Python keyword
- Import module using `import` statement

Example

```
def add(a, b):  
    return a + b
```

```
def subtract(a, b):  
    return a - b
```

```
def multiply(a, b):  
    return a * b
```

```
def divide(a, b):  
    if b != 0:  
        return a / b  
    else:  
        return "Cannot divide by zero"
```

```
import h1  
  
def main():  
    num1=int(input("Enter num1: "))  
    num2=int(input("Enter num2: "))  
    print('the addition is: ',h1.add(num1,num2))  
    print('the subtraction is: ',h1.subtract(num1,num2))  
    print('the multiplication is: ',h1.multiply(num1,num2))  
    print('the division is: ',h1.divide(num1,num2))  
  
main()
```

Menu Driven Programs

- Menu-driven program: displays a list of operations on the screen, allowing user to select the desired operation
 - List of operations displayed on the screen is called a *menu*
- Program uses a decision structure to determine the selected menu option and required operation
 - Typically repeats until the user quits

```
def get_numbers(): 3 usages
    number1 = float(input('Enter number1: '))
    number2 = float(input('Enter number2: '))
    return number1, number2

def add(): 1 usage
    number1, number2 = get_numbers()
    print(f'result: {number1} + {number2} = {number1 + number2}')

def subtraction(): 1 usage
    number1, number2 = get_numbers()
    print(f'result: {number1} - {number2} = {number1 - number2}')

def multiplication(): 1 usage
    number1, number2 = get_numbers()
    print(f'result: {number1} * {number2} = {number1 * number2}')
```

```

import test2
def display(): 1 usage
    print('Menu')
    print('1. Add two numbers')
    print('2. Subtract two numbers')
    print('3. Multiply two numbers')
    print('4. Quit')
def condition(): 1 usage
    again = 'y'
    while again == 'y':
        display()
        choice = input('Enter your choice (1-4): ')
        if choice == '1':
            test2.add()
        elif choice == '2':
            test2.subtraction()
        elif choice == '3':
            test2.multiplication()
        elif choice == '4':
            print('Goodbye')
            break
        else:
            print('Invalid choice, please choose between 1 and 4')
    condition()

```


Conditionally Executing the `main` Function (1 of 3)

- It is possible to create a module that can be run as a standalone program or imported into another program
- Suppose *Program A* defines several functions that you want to use in *Program B*
- So, you import *Program A* into *Program B*
- However, you do not want *Program A* to execute its main function when you import it

Conditionally Executing the `main` Function (2 of 3)

- In the aforementioned scenario, you write each module so it executes its `main` function only when the module is being run as the main program
 - When a source code file is loaded into the Python interpreter, a special variable called `__name__` is created
 - If the source code file has been imported as a module, the `__name__` variable will be set to the name of the module.
 - If the source code file is being executed as the main program, the `__name__` variable will be set to the value `'__main__'`.

Conditionally Executing the `main` Function (3 of 3)

- To prevent the `main` function from being executed when the file is imported as a module, you can conditionally execute `main`

```
def main():  
    statement  
    statement
```

```
def my_function():  
    statement  
    statement
```

```
if __name__ == '__main__':  
    main()
```

Turtle Graphics: Modularizing Code with Functions (1 of 6)

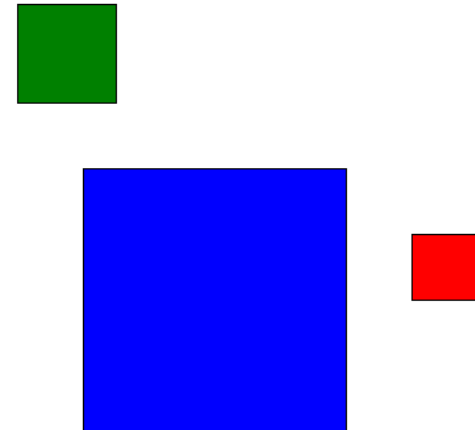
- Commonly needed turtle graphics operations can be stored in functions and then called whenever needed.
- For example, the following function draws a square. The parameters specify the location, width, and color.

```
def square(x, y, width, color):  
    turtle.penup()           # Raise the pen  
    turtle.goto(x, y)        # Move to (X,Y)  
    turtle.fillcolor(color)   # Set the fill color  
    turtle.pendown()         # Lower the pen  
    turtle.begin_fill()       # Start filling  
    for count in range(4):    # Draw a square  
        turtle.forward(width)  
        turtle.left(90)  
    turtle.end_fill()         # End filling
```

Turtle Graphics: Modularizing Code with Functions (2 of 6)

- The following code calls the previously shown `square` function to draw three squares:

```
square(100, 0, 50, 'red')  
square(-150, -100, 200, 'blue')  
square(-200, 150, 75, 'green')
```



Turtle Graphics: Modularizing Code with Functions (3 of 6)

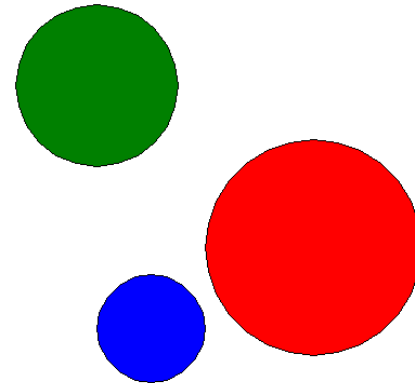
- The following function draws a circle. The parameters specify the location, radius, and color.

```
def circle(x, y, radius, color):  
    turtle.penup()                # Raise the pen  
    turtle.goto(x, y - radius)    # Position the turtle  
    turtle.fillcolor(color)       # Set the fill color  
    turtle.pendown()              # Lower the pen  
    turtle.begin_fill()           # Start filling  
    turtle.circle(radius)         # Draw a circle  
    turtle.end_fill()             # End filling
```

Turtle Graphics: Modularizing Code with Functions (4 of 6)

- The following code calls the previously shown `circle` function to draw three circles:

```
circle(0, 0, 100, 'red')  
circle(-150, -75, 50, 'blue')  
circle(-200, 150, 75, 'green')
```



Turtle Graphics: Modularizing Code with Functions (5 of 6)

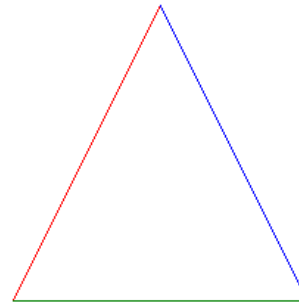
- The following function draws a line. The parameters specify the starting and ending locations, and color.

```
def line(startX, startY, endX, endY, color):  
    turtle.penup()           # Raise the pen  
    turtle.goto(startX, startY) # Move to the starting point  
    turtle.pendown()         # Lower the pen  
    turtle.pencolor(color)    # Set the pen color  
    turtle.goto(endX, endY)   # Draw a square
```


Turtle Graphics: Modularizing Code with Functions (6 of 6)

- The following code calls the previously shown `line` function to draw a triangle:

```
TOP_X = 0
TOP_Y = 100
BASE_LEFT_X = -100
BASE_LEFT_Y = -100
BASE_RIGHT_X = 100
BASE_RIGHT_Y = -100
line(TOP_X, TOP_Y, BASE_LEFT_X, BASE_LEFT_Y, 'red')
line(TOP_X, TOP_Y, BASE_RIGHT_X, BASE_RIGHT_Y, 'blue')
line(BASE_LEFT_X, BASE_LEFT_Y, BASE_RIGHT_X, BASE_RIGHT_Y, 'green')
```



Summary (1 of 2)

- This chapter covered:
 - The advantages of using functions
 - The syntax for defining and calling a function
 - Methods for designing a program to use functions
 - Use of local variables and their scope
 - Syntax and limitations of passing arguments to functions
 - Global variables, global constants, and their advantages and disadvantages

Summary (2 of 2)

- Value-returning functions, including:
 - Writing value-returning functions
 - Using value-returning functions
 - Functions returning multiple values
- Using library functions and the `import` statement
- Modules, including:
 - The `random` and `math` modules
 - Grouping your own functions in modules
- Modularizing Turtle Graphics Code